

CSE 256 | Lecture 14: LLM Scaling Laws + Efficient Adaptation

Ndapa Nakashole

February 19, 2026

Administrative matters

- ▶ **PA 1 grades:** have been released, check Gradescope.
- ▶ **PA 2:** due on tomorrow Friday Feb 20th - can submit up to 2 days late with penalty as described in the syllabus.

Recap: Decoding Strategies

- ▶ LMs place a distribution $P(x_i \mid x_1, \dots, x_{i-1})$. How do we generate text from these?
- ▶ Maximization-based decoding:
 - ◉ **Greedy decoding**
 - ◉ **Beam search**
- ▶ Sampling-based decoding:
 - ◉ **Random sampling**
 - ◉ **Nucleus sampling**
 - ◉ **Top- k sampling** (optionally with temperature)

Today

- ① LLM Scaling Laws
- ② Efficient Adaptation (part 1)
 - ◉ Zero-shot and few-shot prompting
 - ◉ Chain-of-Thought (CoT) prompting
- ③ Post-training
 - ◉ Instruction tuning
 - ◉ Reinforcement learning with human feedback (RLHF)

LLM Scaling Laws

Scaling Laws Motivation

- ▶ Training large models is extremely expensive. Hyperparameter tuning multiplies that cost.
- ▶ How to solve this problem?
 - ◉ Guess and pray
 - ◉ Exhaustive grid search
 - ◉ **Discover simple scaling rules**

Carbon Footprint of Training Large Models

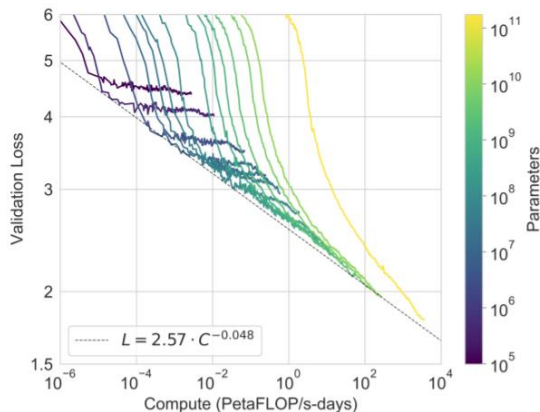
Consumption	CO₂e (lbs)
Air travel, 1 passenger, NY↔SF	1984
Human life, avg, 1 year	11,023
American life, avg, 1 year	36,156
Car, avg incl. fuel, 1 lifetime	126,000
Training one model (GPU)	
NLP pipeline (parsing, SRL)	39
w/ tuning & experimentation	78,468
Transformer (big)	192
w/ neural architecture search	626,155

Table 1: Estimated CO₂ emissions from training common NLP models, compared to familiar consumption.¹

What are Scaling “Laws”?

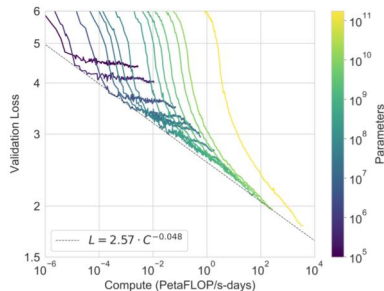
Simple, predictive “laws” for language model performance as a function of scale (parameters, data, compute)

- ▶ **Old:** tune hyperparameters on big models
- ▶ **New:** tune on small models, extrapolate to large ones



Scaling Laws: Compute vs Test Loss

- ▶ Best achievable **validation loss** as a function of total **training compute**
- ▶ Models are:
 - ◉ Trained to convergence
 - ◉ Given sufficient data
- ▶ Each curve = different model size



Dashed line:

Optimal frontier: the lowest loss achievable at each compute budget

Scaling Laws as a Design Tool

- ▶ How should we design large language models under finite compute budgets?
- ▶ **Architectural choices**
 - ◉ LSTMs vs Transformers
 - ◉ Depth vs width
 - ◉ Cosine vs linear learning rate decay
 - ◉ ... (many more)
- ▶ **Resource allocation trade-offs**
 - ◉ More data vs more GPUs?
 - ◉ ... (many more)
- ▶ Scaling laws turn these into **predictable optimization problems**

Scaling Laws for Transformer Language Models

- ▶ Suppose we have a fixed compute budget:

$$C \approx N \times D$$

where

- ◉ N = number of parameters
 - ◉ D = number of training tokens
- ▶ **What is the optimal allocation of compute between N and D ?**
 - ▶ We want an answer that:
 - ◉ Generalizes to much larger scales
 - ◉ Requires only small-scale experiments

Thought Experiment: Fixed Compute Budget

You have one GPU for 24 hours.

How should you allocate compute between:

- ▶ Model size (N)
 - ▶ Dataset size (D)
-
- 1 Small model, huge dataset (100K parameters, 5K books)
 - 2 Medium model, medium dataset (5M parameters, 100 books)
 - 3 Huge model, tiny dataset (50M parameters, 1 book)

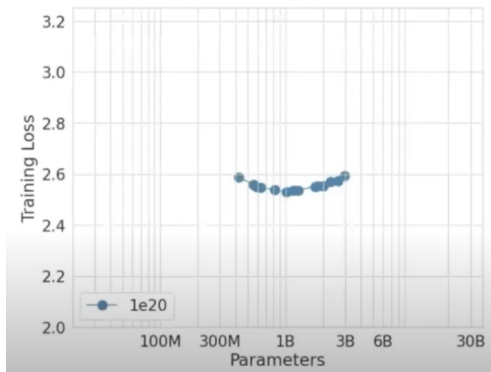
Which minimizes test loss?

Chinchilla Scaling Laws Approach

Revisit: Given a fixed FLOPs budget, how should one trade-off model size and the number of training tokens?

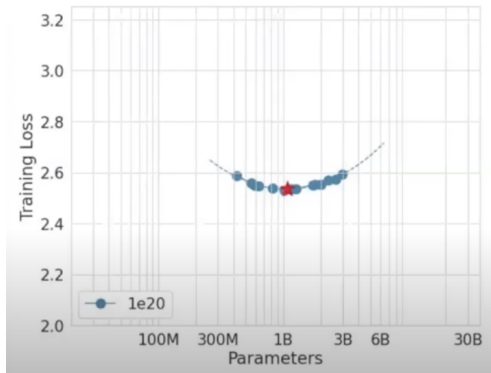
Approach: Fix FLOPs and vary model size and training tokens

- 1 Fix a target FLOPs budget
- 2 Train a few models, vary model size



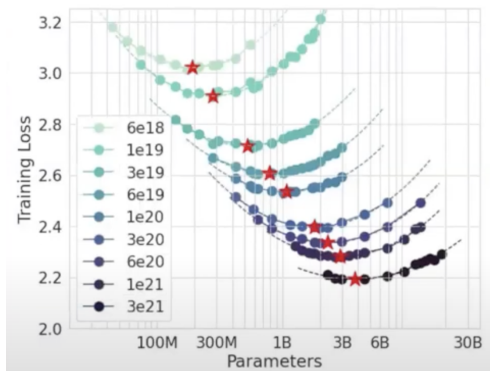
Fix FLOPs and vary model size and training tokens

- 1 Fix a target FLOPs budget
- 2 Train a few models, vary model size (we choose training tokens such that the final FLOPs is a constant)
- 3 Fit a parabola and take the minimum



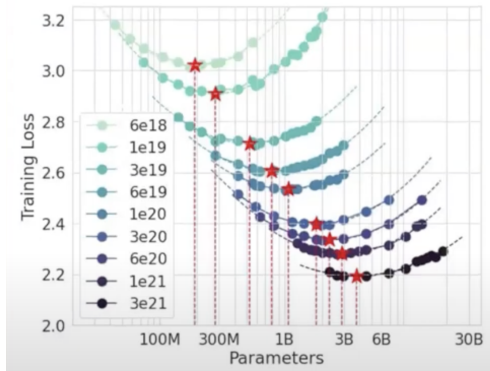
Fix FLOPs and vary model size and training tokens

- 1 Fix a target FLOPs budget
- 2 Train a few models, vary model size
- 3 Fit a parabola and take the minimum
- 4 Repeat steps 1 – 3 for various FLOPs budgets



Fix FLOPs and vary model size and training tokens

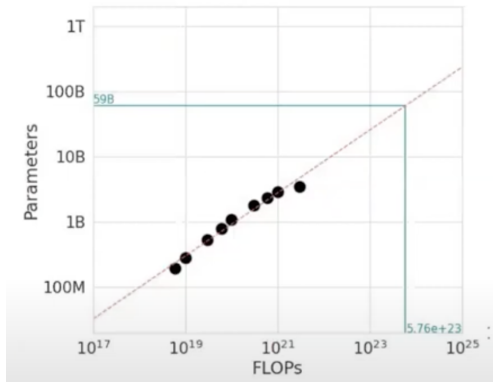
- 1 Fix a target FLOPs budget
- 2 Train a few models, vary model size
- 3 Fit a parabola and take the minimum
- 4 Repeat steps 1 – 3 for various FLOPs budgets



Can now extrapolate parameter count for each FLOPs budget

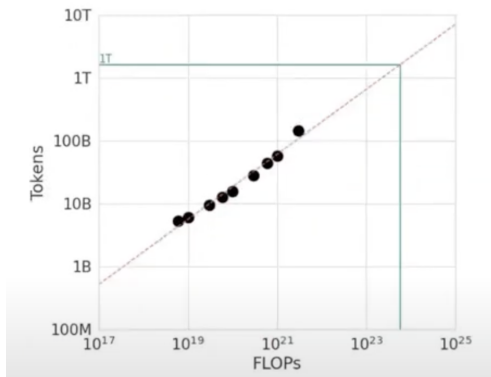
IsoFlops: Fix flops and vary model size and training tokens

- 5 Fit a power law between FLOPs budget and optimal model size
- 6 Plot on log-log axes, fit a line, and extrapolate to larger FLOPs budgets



Fix flops and vary model size and training tokens

- 1 Do the same thing, but fit between FLOPs budget and optimal token count



Chinchilla vs Prior Large Models

Same FLOPs budget as Gopher (280B).

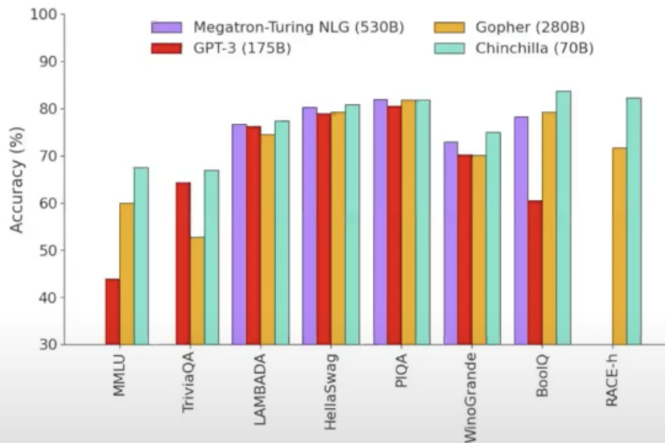
Model	Parameters	Training Tokens
GPT-3	175B	300B
Gopher	280B	320B
Chinchilla	70B	1.4T

Key difference:

Tokens per parameter

- ▶ GPT-3: ~ 1.7 tokens / parameter
- ▶ Chinchilla: ~ 20 tokens / parameter

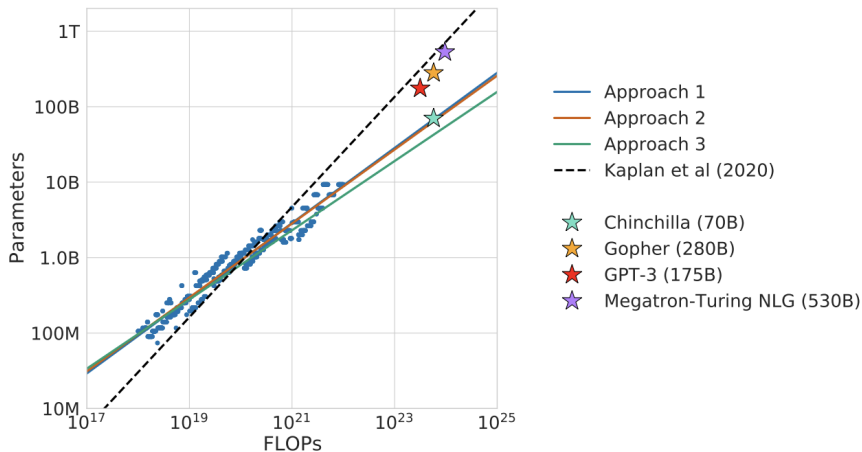
Chinchilla: Downstream Performance



Observation:

A 70B model outperforms a 280B model when trained with optimal data scaling.

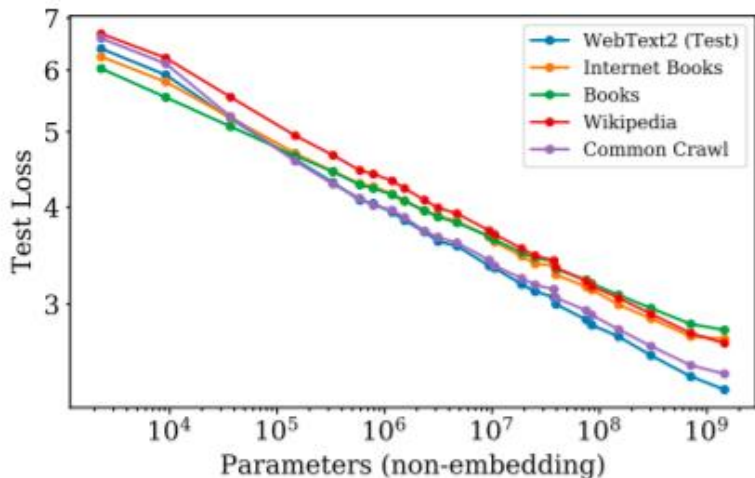
Many Large Models Were Undertrained



Frontier LLMs were too large for the amount of data they were trained on.

Scaling laws: surprisingly clean and robust

They also hold in non-standard settings (when train $\neq$ test)

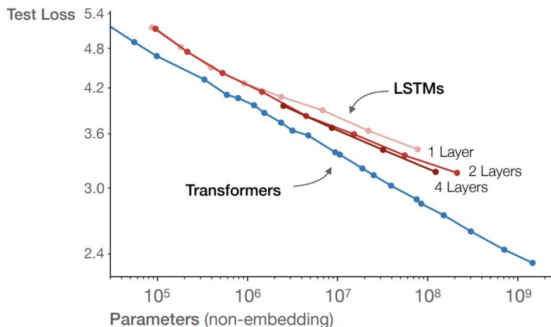


Cross-model: transformers vs LSTMs

Q: Are transformers better than LSTMs?

Brute force way: spend tens of millions to train a LSTM GPT-3

Scaling law way:



[Kaplan+ 2021]

Some surprising takeaways

- ▶ The effect of many hyperparameters on big LMs can be predicted before training!
 - ◉ Optimizer choice
 - ◉ Model depth
 - ◉ Architecture choice

Practical recipe:

- 1 Train several small-scale models with different hyperparameters.
- 2 Fit scaling curves for each configuration.
- 3 Compare extrapolated large-scale performance.

Goal: Choose the configuration with the best asymptotic scaling.

Scaling Laws Assumptions

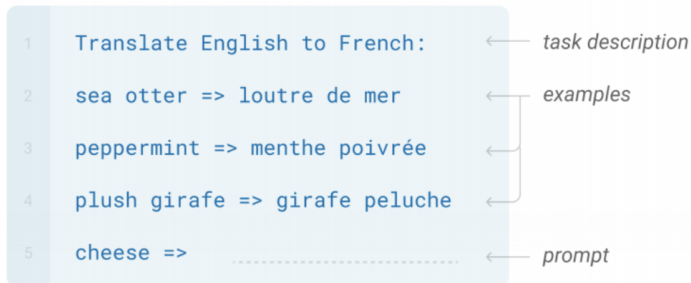
- ① The compute vs optimal model size can be modelled as a power law
 - ⊙ Simple model that fits well with empirical evidence
 - ⊙ Linear relationship in log-log space
- ② "All tokens are created equal": We train on a fixed uniform dataset
 - ⊙ Very unlikely to be true, but matches current training of LLMs
 - ⊙ Training is in the < 1 epoch regime
- ③ The model is fixed: Auto-regressive Transformer
 - ⊙ Unlikely to be the "optimal" model
 - ⊙ The best model currently available

Efficient Adaptation of LLMs to New Tasks:

- ▶ Zero-shot and few-shot prompting (in-context learning)
 - ◉ Specify the task at inference time without updating model parameters

GPT-3: Few-Shot Learning Without Fine-Tuning

- ▶ Fine-tuning a 175B model is:
 - ⊙ Expensive; Task-specific; Requires gradient updates
- ▶ GPT-3 proposes an alternative: **In-context learning**
 - ⊙ No parameter updates; Task specified at inference time; Adaptation through the prompt



GPT-3 Few-Shot Results (SuperGLUE)

Main takeaway: A 175B parameter model, with *no gradient updates*, achieves competitive performance on many tasks.

	SuperGLUE Avg	COPA Acc	ReCoRD F1
Fine-tuned SOTA	89.0	94.8	93.3
Fine-tuned BERT-Large	69.0	70.6	72.0
GPT-3 Few-Shot	71.8	92.0	91.1

- ▶ Sometimes close to fine-tuned SOTA
- ▶ Sometimes much worse
- ▶ Remarkable because: **no task-specific training**

From Pretraining to Prompting

- ▶ Large language models are trained to:

predict the next token

- ▶ They are *not* trained explicitly for:
 - ◉ Sentiment classification
 - ◉ Translation
 - ◉ Question answering
- ▶ Key idea: **If we can phrase a task as next-token prediction**, the pretrained model can attempt it.

Zero-shot Prompting for Text Classification

- ▶ Single unlabeled datapoint x , want to predict label y
 $x =$ The movie's acting could've been better
- ▶ Wrap x in a template

Prompt:

Review: The movie's acting could've been better
On a 1 to 4 star scale, the reviewer would probably give
this movie _ _ _



2 stars

Two Ways to Use an LM for Classification

► 1. Generate and read the answer

- ◉ Prompt: "This review is ____."
- ◉ Let the model complete the sentence.
- ◉ Problem: the model may be **verbose** or inconsistent.
- ◉ Example: "I would rate this movie 2 stars."

► 2. Compare label probabilities (constrained decoding)

- ◉ Prompt: "Out of positive, neutral, or negative, this review is ____."
- ◉ Compute (need access to model probabilities):

$$P(\text{positive} \mid \text{context}), P(\text{neutral} \mid \text{context}), \dots$$

- ◉ Choose the highest-probability label.

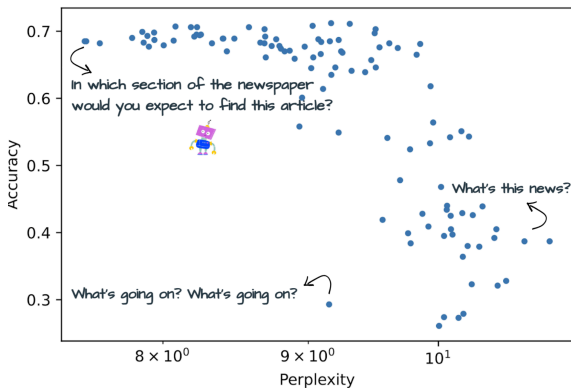
Prompt Variability Matters

- ▶ Many different prompts for the same task:

- Manual writing
- Paraphrasing
- Backtranslation

- ▶ x -axis: Prompt perplexity (How natural under the pretrained model?)

- ▶ y -axis: Task performance



Small wording changes can cause large performance swings.

Prompt Variability Is Large

- ▶ For OPT-175B:
 - ⊙ Average over all prompts: moderate performance
 - ⊙ Average over best 50% of prompts: much better

Task	Avg Acc	Acc 50%
Antonyms	—	—
GLUE Cola	47.7	57.1
Newspop	66.4	72.9
AG News	57.5	68.7
IMDB	86.2	91.0
DBpedia	46.7	55.2
Emotion	16.4	23.0
Tweet Offensive	51.3	55.8

Few-shot prompting

- Provide a few examples of the task together with the instruction

Prompt:

Please classify movie reviews as 'positive' or 'neutral' or 'negative'.

Input: I really don't like this movie.

Output: negative

Input: This movie is great!

Output: positive

Input: The movie's acting could've been better.

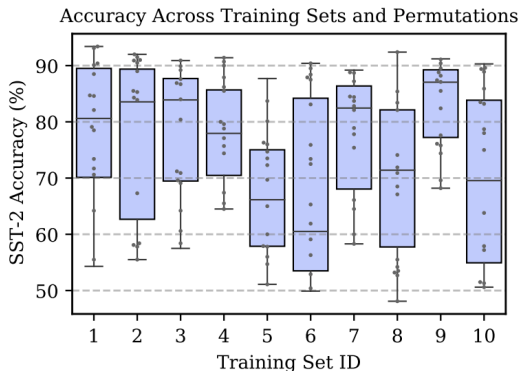
Output:



negative

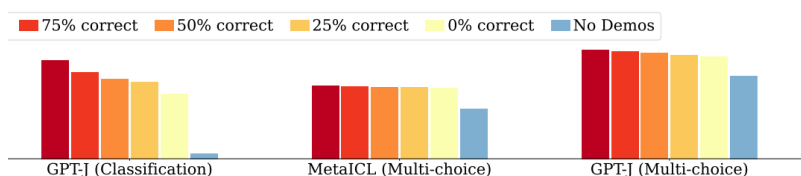
Few-Shot Prompting: What Can Go Wrong?

- ▶ Training sets
- ▶ Different orderings (permutations) of the same examples.
- ▶ Large performance variance across example sets and orderings.



Key takeaway: Few-shot performance is sensitive to example choice and ordering.

Rethinking Demonstrations: Counter-Intuitive Effects



- ▶ Are the demonstrations teaching the task?
- ▶ Surprising findings:
 - ◉ Random labels do not drastically reduce performance.
 - ◉ Mislabeled demonstrations can outperform having no demonstrations.
- ▶ Suggests: **The structure and format of the prompt matter, not just the correctness of labels.**

Prompting: Summary

- ▶ Prompting enables **task adaptation at inference time**
- ▶ Zero-shot:
 - ◉ Highly sensitive to wording
 - ◉ Prompt design can dramatically change performance
- ▶ Few-shot:
 - ◉ Demonstrations reduce ambiguity
 - ◉ But performance depends on example choice and order
- ▶ Prompting is powerful, but not fully robust.

Chain-of-Thought Prompting

A prompting technique for multi-step reasoning

Chain-of-Thought (CoT) Prompting

- For multi-step problems, we can ask the model to generate **intermediate steps** before the final answer.
 - Often improves accuracy on arithmetic, logic, and commonsense reasoning.

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

[[Wei et al., 2022](#); also see [Nye et al., 2021](#)]

Zero-Shot Chain-of-Thought Prompting

- ▶ Do not need few-shot examples in CoT prompting
- ▶ Simply adding a reasoning cue can improve performance:
"Let's think step by step".

(a) Few-shot

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The answer is 8. ✗

(b) Few-shot-CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A:

(Output) The juggler can juggle 16 balls. Half of the balls are golf balls. So there are $16 / 2 = 8$ golf balls. Half of the golf balls are blue. So there are $8 / 2 = 4$ blue golf balls. The answer is 4. ✓

(c) Zero-shot

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

(d) Zero-shot-CoT (Ours)

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

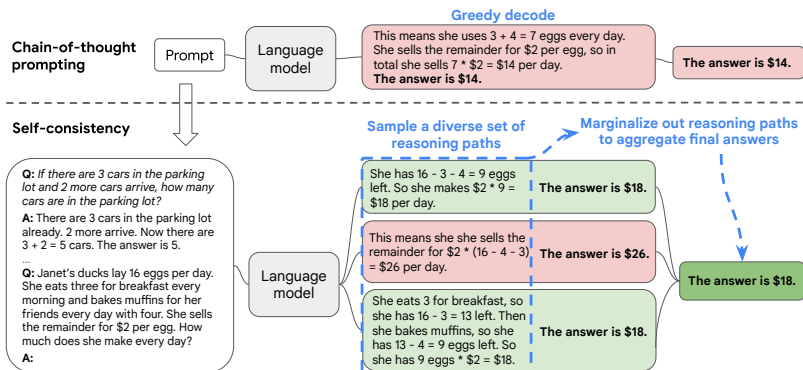
A: **Let's think step by step.**

(Output) There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓

~~Instruction-tuned LLMs often reason even without this explicit cue.~~

Self-Consistency in Chain-of-Thought Prompting

- ▶ Instead of generating one chain of thought, sample **multiple reasoning paths**.
- ▶ Take a majority vote over the final answers.
- ▶ Idea: correct reasoning paths are more likely to agree.



Chain-of-Thought Prompting: Summary

- ▶ Asking for **intermediate reasoning steps** can improve accuracy.
 - ◉ Especially for multi-step reasoning:
 - ▶ Math problems
 - ▶ Multi-hop Question Answering
 - ▶ Commonsense reasoning
- ▶ Self-consistency:
 - ◉ Sample multiple reasoning paths
 - ◉ Aggregate answers (majority vote)
 - ◉ Reduces stochastic reasoning errors
- ▶ CoT improves performance *without changing model parameters*.

Instruction Tuning

Instead of prompting tricks to get better performance, we can also **train** the model to follow instructions better.

Language Modeling $\neq$ Assisting Users

Base LLM

Predicts next word, based on
text training data

Once upon a time, there was a unicorn
that lived in a magical forest with
all her unicorn friends

What is the capital of France?
What is France's largest city?
What is France's population?
What is the currency of France?

Problem:

- ▶ Base LLMs are trained for next-token prediction
- ▶ They are *not* trained to follow instructions.
- ▶ Prompt tricks are powerful, but brittle.

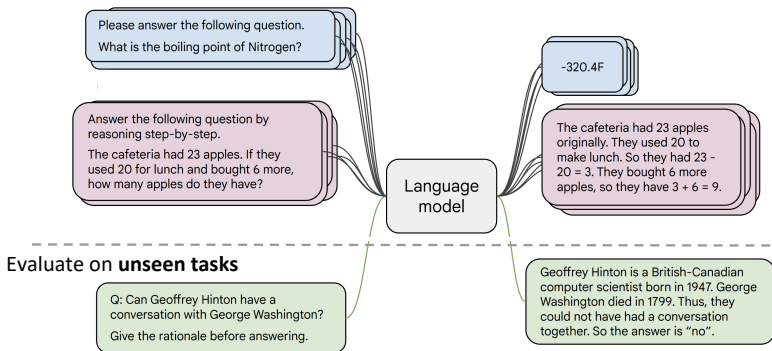
Instruction Tuned LLM

What is the capital of France?

The capital of France is Paris.

Instruction Fine-Tuning

Collect examples of (instruction, output) pairs across many tasks and finetune an LM



[FLAN-T5; [Chung et al., 2022](#)]

- ▶ Supervised fine-tuning on (instruction, response) pairs
- ▶ Optionally followed by preference optimization (e.g., RLHF)

Limitations of Instruction Tuning

Motivation for RLHF

- ▶ **Ground-Truth Assumption:** Supervised fine-tuning requires high-quality (instruction, response) pairs.
 - ⊙ Expensive to collect at scale
- ▶ **Ambiguity in Open-Ended Tasks:** Many tasks have no single correct answer.
 - ⊙ Example: *"Write a story about a dog and her pet grasshopper."*
- ▶ **Likelihood $\neq$ Preference:** Token-level likelihood treats all errors equally.
 - ⊙ But some mistakes matter more than others.
 - ⊙ Human judgments are holistic, not token-level.

Directly optimize for human preferences?

Optimizing for Human Preferences

► Reinforcement Learning from Human Feedback (RLHF)

- ◉ Learn a reward model from human preference comparisons
- ◉ Optimize the language model to maximize that reward

SAN FRANCISCO,
California (CNN) --
A magnitude 4.2
earthquake shook the
San Francisco
...
overturn unstable
objects.

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

$$\begin{matrix} s_1 \\ R(s_1) = 8.0 \end{matrix}$$

The Bay Area has
good weather but
prone to
earthquakes and
wildfires.

$$\begin{matrix} s_2 \\ R(s_2) = 1.2 \end{matrix}$$

$$\max_{\pi_{\theta}} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [R_{\phi}(x, y)]$$

Reinforcement Learning for Instruction Following

- ▶ In **Reinforcement Learning (RL)**, we do not provide a single ground-truth answer.
- ▶ The model (policy) generates its own response:

$$y \sim \pi_{\theta}(\cdot \mid x)$$

- ▶ A **reward function** evaluates the response:

$$R_{\phi}(x, y)$$

- ⊙ Often learned from human preference comparisons
- ▶ The objective:

$$\max_{\pi_{\theta}} \mathbb{E}_{y \sim \pi_{\theta}} [R_{\phi}(x, y)]$$

InstructGPT: scaling up RLHF

1) Instruction Fine-tuning; 2) Human preferences from comparison data; 3) Optimize a policy against a reward model using RL

Step 1

Collect demonstration data, and train a supervised policy.

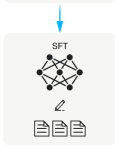
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

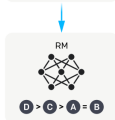
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

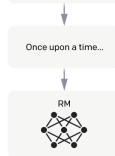


The policy generates an output.

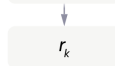


Once upon a time...

The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Why Both Instruction-Tuning & RLHF?

Instruction-Tuning (SFT)

- ▶ Learns from **demonstrations** (positive examples only)
- ▶ Optimizes likelihood of provided answers
- ▶ May encourage confident but incorrect responses
- ▶ No explicit signal for “bad” outputs

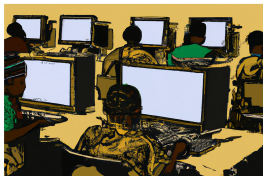
RLHF

- ▶ Learns from **preferences** (good vs bad comparisons)
- ▶ Optimizes a **reward signal**
- ▶ Penalizes undesirable behaviors
- ▶ Encourages calibrated responses or abstention

Where do the labels come from?

Exclusive: OpenAI Used Kenyan Workers on Less Than \$2 Per Hour to Make ChatGPT Less Toxic

15 MINUTE READ



STORY HUB REVERIES 15.10.2023 00:00 AM

Millions of Workers Are Training AI Models for Pennies

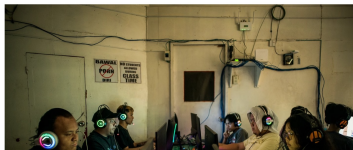
From the Philippines to Colombia, low-paid workers label training data for AI models used by the likes of Amazon, Facebook, Google, and Microsoft



Connecting with Facebook with her dog. COURTESY OF SHARIN JESS PATRICK

Behind the AI boom, an army of overseas workers in 'digital sweatshops'

By Rebecca Tan and Roshan Subedi
August 20, 2023 at 2:00 a.m. EDT



RLHF labels are often obtained from overseas, low-wage workers

Instruction-Tuning and RLHF: Summary

► Instruction-Tuning + RLHF

- ◉ Transform base LLMs into usable assistants
- ◉ Align outputs with human instructions and preferences

► But alignment is not solved:

- ◉ Hallucinations may still occur
- ◉ Model knowledge is still limited by training data

► Reducing Hallucinations:

- ◉ Retrieve relevant information first
- ◉ Ground generation in external sources
- ◉ → Retrieval-Augmented Generation (RAG) = a coming up lecture

Questions?